

C++ Move Semantics Study Guide

Improved Version With Properly Formatted Code Examples

Table of Contents

1. Why Move Semantics Exists
2. Lvalues and Rvalues
3. Reference Types
4. Why const T& Accepts Temporaries
5. Why T&& Was Added
6. Named Rvalue References
7. std::move Explained
8. Move Constructor
9. Move Assignment
10. Rule of Five and Rule of Zero
11. Move-Only Types
12. RVO and NRVO
13. noexcept
14. std::forward
15. Common Mistakes
16. Complete Examples

Why Move Semantics Exists

Move semantics avoids expensive copies by transferring ownership of resources such as heap memory, file handles, sockets, threads, GPU buffers, and camera frame buffers.

Lvalues and Rvalues

Lvalues have names and identity. Rvalues are temporary expressions or temporary objects.

Reference Types

T& binds to lvalues only. const T& binds to both lvalues and temporaries. T&& binds to rvalues.

Why const T& Accepts Temporaries

A const reference promises not to modify the object, so C++ safely extends the temporary's lifetime.

Why T&& Was Added

T&& was introduced in C++11 to allow efficient ownership transfer from temporary objects.

Named Rvalue References

Even if a variable has type T&&, once it has a name it becomes an lvalue expression.

std::move Explained

std::move does not move anything. It performs a cast that allows move operations to be selected.

Move Constructor

Used to construct a new object by stealing resources from another object.

Move Assignment

Used to transfer ownership into an already-existing object.

Example 1: Reference Binding Rules

```
#include <string>

std::string name = "Farouq";

foo(name);           // lvalue
foo(std::string("temp")); // rvalue

// Reference types

std::string& ref1 = name;           // OK
const std::string& ref2 = name;     // OK
const std::string& ref3 = std::string("x"); // OK

// std::string& ref4 = std::string("x"); // ERROR

std::string&& ref5 = std::string("x"); // OK
```

Example 2: Named Rvalue References

```
#include <iostream>
#include <string>

void test(std::string&)
{
    std::cout << "lvalue\n";
}

void test(std::string&&)
{
    std::cout << "rvalue\n";
}
```

```

}

int main()
{
    std::string&& s = std::string("hello");

    test(s);                // calls lvalue version

    test(std::move(s));    // calls rvalue version
}

```

Expected Output

```

lvalue
rvalue

```

Example 3: Move Constructor

```

class Buffer
{
public:

    Buffer(Buffer&& other) noexcept
    {
        data_ = other.data_;

        other.data_ = nullptr;
    }

private:

    int* data_;
};

Buffer b1;

Buffer b2 = std::move(b1);

```

Example 4: Why return std::move(local) is Usually Wrong

```

std::string makeName()
{
    std::string name = "Farouq";

    return name;                // Preferred
}

std::string makeName2()
{
    std::string name = "Farouq";

    return std::move(name);    // Usually worse
}

```

Interview Cheat Sheet

1. Move transfers ownership.
2. `std::move` is a cast.
3. Lvalues have names.
4. Rvalues are temporaries.
5. `T&` binds to lvalues.
6. `const T&` binds to both.
7. `T&&` binds to rvalues.
8. Named rvalue references are lvalues.
9. Moved-from objects are valid but unspecified.
10. Prefer `'return obj;'` over `'return std::move(obj);'`